

RÉPUBLIQUE DU CAMEROUN

Paix - Travail - Patrie

UNIVERSITÉ DE YAOUNDÉ I

FACULTÉ DES SCIENCES

DÉPARTEMENT D'INFORMATIQUE



REPUBLIC OF CAMEROON

Peace - Work - Fatherland

UNIVERSITY OF YAOUNDE I

FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER
SCIENCE

INF 4238 : Vision par Ordinateur

Rapport d'Exposé

Analyse de l'Intensité et Histogrammes :

Technique d'égalisation et de normalisation pour la robustesse
des descripteurs globaux.

Rédigé et présenté par le groupe 8 :

DASSI MANDJO LEA JUSTINE

22W2164

MELONG LETHYCIA

22W2147

NGUEFACK TANGOMO CHRIS ARTHUR

25G2055

Projet disponible sur :

<https://github.com/leadassi/INF4038>

Sous la supervision du :

Dr. TAPAMO KENFACK Hippolyte

Enseignant à l'Université de Yaoundé I

Année académique : 2025-2026

Table des matières

1	Introduction	2
2	Fondements Théoriques	2
2.1	Modèle d'Intensité d'une Image Numérique	2
2.2	L'Histogramme d'Image	2
2.2.1	Histogramme Normalisé (PDF)	2
2.2.2	Fonction de Répartition Cumulative (CDF)	3
3	Techniques d'Égalisation d'Histogramme	3
3.1	Égalisation Globale (GHE)	3
3.1.1	Principe	3
3.1.2	Effets mécaniques	3
3.1.3	Limites de la GHE	3
3.2	Égalisation Adaptative Limitée par Contraste (CLAHE)	4
3.2.1	Algorithme CLAHE	4
3.2.2	Comparaison GHE vs CLAHE	4
4	Techniques de Normalisation d'Intensité	5
4.1	Normalisation Min-Max	5
4.2	Normalisation Z-Score (Standardisation)	5
4.3	Normalisation L^2 sur les Vecteurs de Descripteurs	5
5	Descripteurs Globaux et Robustesse	6
5.1	Définition et Types de Descripteurs Globaux	6
5.1.1	Histogramme de Couleurs	6
5.1.2	LBP — Local Binary Pattern (global)	6
5.1.3	HOG — Histogramme des Gradients Orientés	6
5.2	Mesures de Similarité	6
5.3	Pipeline Complet de Traitement	7
6	Implémentation	7
6.1	Architecture de l'Application	7
6.2	Bibliothèques Utilisées	7
6.3	Fonctions Principales	8
6.4	Captures d'Écran de l'Application	11
7	Conclusion	13
7.1	Synthèse des Résultats	13
7.2	Perspectives	13

1. Introduction

En vision par ordinateur, l'une des difficultés fondamentales réside dans la **variabilité des conditions d'acquisition**. Une même scène photographiée sous des éclairages différents produit des matrices de pixels aux valeurs numériques très divergentes, bien que le contenu sémantique soit identique. Ce phénomène constitue un obstacle majeur pour tout système de reconnaissance ou de classification visuelle basé sur des descripteurs globaux.

Problématique centrale : Comment extraire des caractéristiques visuelles (descripteurs) qui restent **invariantes aux conditions d'acquisition**, notamment aux variations d'éclairage, tout en demeurant discriminantes pour distinguer des scènes différentes ?

Ce travail répond à cette problématique en explorant trois familles de techniques de pré-traitement d'image :

- L'**analyse de l'histogramme d'intensité** comme outil de diagnostic
- Les **techniques d'égalisation** : GHE (globale) et CLAHE (adaptative)
- Les **techniques de normalisation** : Min-Max, Z-Score, et normalisation L^2 des descripteurs

2. Fondements Théoriques

2.1. Modèle d'Intensité d'une Image Numérique

Une image numérique est formalisée comme une fonction discrète :

$$I : \Omega \subset \mathbb{Z}^2 \longrightarrow \{0, 1, \dots, L - 1\}$$

où $\Omega = \{0, \dots, M-1\} \times \{0, \dots, N-1\}$ est le domaine spatial de taille $M \times N$, et L est le nombre de niveaux d'intensité. Pour une image 8 bits, $L = 2^8 = 256$, avec la convention $0 \equiv$ noir et $255 \equiv$ blanc.

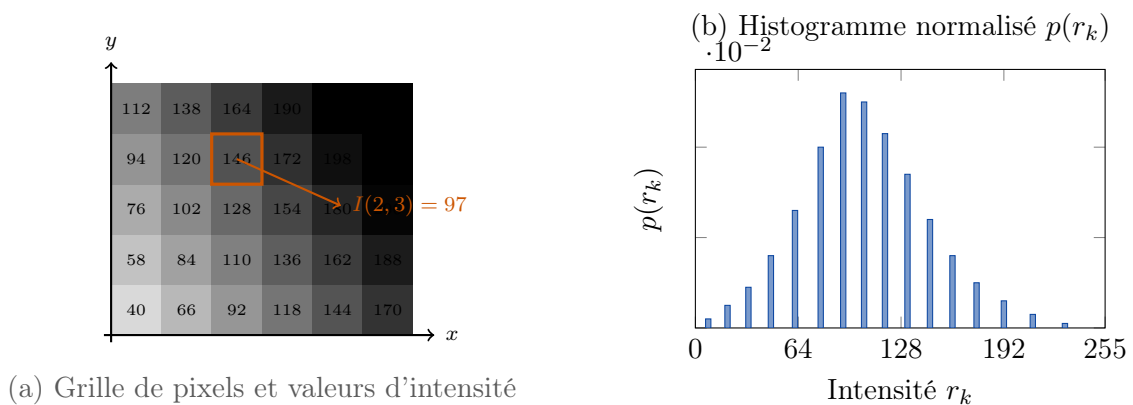


FIGURE 1 – Modèle d'intensité d'une image numérique

2.2. L'Histogramme d'Image

2.2.1. Histogramme Normalisé (PDF)

L'histogramme normalisé est défini pour chaque intensité r_k par :

$$p(r_k) = \frac{n_k}{M \times N}, \quad \forall k \in \{0, 1, \dots, L - 1\}$$

où n_k est le nombre de pixels d'intensité r_k . La somme des fréquences est unitaire : $\sum_{k=0}^{L-1} p(r_k) = 1$.

La fonction $p(r_k)$ constitue une estimation de la **Densité de Probabilité (PDF)** des niveaux de gris. Elle caractérise globalement le contraste et permet de détecter les sous-expositions, sur-expositions ou faibles dynamiques.

2.2.2. Fonction de Répartition Cumulative (CDF)

La CDF est définie par :

$$F(r_k) = \sum_{j=0}^k p(r_j) = \frac{1}{M \times N} \sum_{j=0}^k n_j, \quad F(r_{L-1}) = 1$$

La CDF est l'outil mathématique central de l'égalisation d'histogramme, car elle permet de construire la transformation optimale d'intensité.

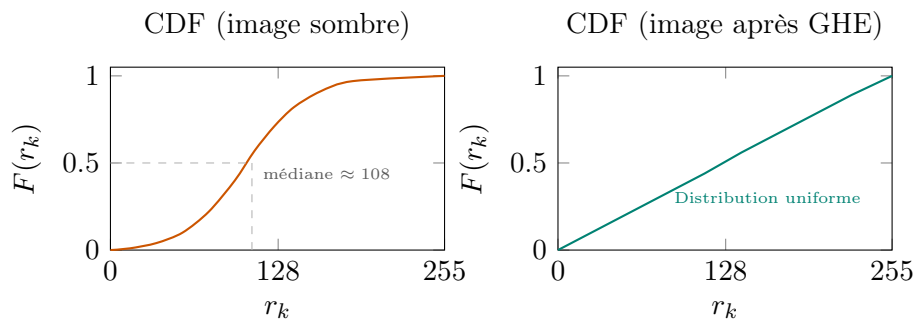


FIGURE 2 – CDF avant et après égalisation globale (GHE)

3. Techniques d'Égalisation d'Histogramme

3.1. Égalisation Globale (GHE)

3.1.1. Principe

L'objectif de la GHE est d'obtenir une distribution uniforme des intensités via une transformation monotone croissante $s = T(r)$. La transformation idéale repose sur la CDF :

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p(r_j) = \frac{L - 1}{M \times N} \sum_{j=0}^k n_j$$

Propriété fondamentale : Si X est une variable aléatoire continue de CDF F_X , alors $Y = F_X(X) \sim \mathcal{U}[0, 1]$. Dans le cas discret, cette uniformité n'est qu'approchée.

3.1.2. Effets mécaniques

- **Étirement de la dynamique :** redistribue les intensités sur tout l'intervalle $[0, L - 1]$
- **Propriété d'invariance :** robustesse accrue face aux décalages uniformes de luminosité (changements de biais additifs)

3.1.3. Limites de la GHE

En traitant l'image **globalement et uniformément**, la GHE présente trois limites importantes :

- **Amplification du bruit** : dans les zones uniformes, de légères variations thermiques du capteur sont sur-amplifiées, créant du grain visible
- **Effet de wash-out** : si l'image contient simultanément des zones très éclairées et des zones sombres, la GHE compresse et sature les textures intermédiaires
- **Ignorance spatiale** : traite l'image comme un ensemble de pixels déconnectés de leur contexte géométrique local

3.2. Égalisation Adaptative Limitée par Contraste (CLAHE)

Pour pallier les défauts de la GHE, la méthode **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*), introduite par Zuiderveld (1994), introduit une logique spatio-contextuelle.

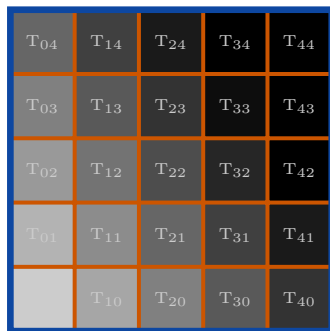
3.2.1. Algorithme CLAHE

1. **Découpage en tuiles** : l'image est segmentée en N_t petites fenêtres locales non chevauchantes (typiquement des blocs de 8×8 pixels)
2. **Écrêtage de l'histogramme local** : chaque histogramme local est tronqué à une hauteur seuil β (le *clip limit*) :

$$\tilde{h}(k) = \min(h(k), \beta)$$

Les pixels excédentaires $\Delta = \sum_k \max(0, h(k) - \beta)$ sont redistribués uniformément sur tous les niveaux

3. **Égalisation locale** : application de la transformation CDF dans chaque tuile
4. **Interpolation bilinéaire** : lissage des transitions aux frontières des tuiles pour éliminer les artefacts de bloc lors de la reconstruction



(a) Division en tuiles 5×5

(b) Écrêtage de l'histogramme local

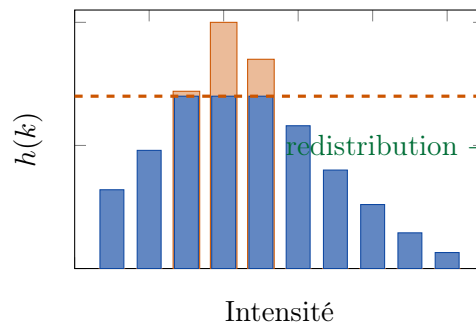


FIGURE 3 – Illustration du découpage en tuiles et de l'écrêtage CLAHE

3.2.2. Comparaison GHE vs CLAHE

Critère	Sans traitement	GHE	CLAHE
Contraste global	faible	amélioré	amélioré
Contraste local	faible	partiel	optimal
Amplification du bruit	nulle	importante	limitée
Préservation des textures	—	dégradée	préservée
Artefacts de bloc	non	possibles	réduits
Complexité	$O(1)$	$O(MN)$	$O(MN)$
Application typique	—	expositions simples	médical, vision nocturne

4. Techniques de Normalisation d'Intensité

4.1. Normalisation Min-Max

La normalisation Min-Max ramène les intensités dans l'intervalle $[0, 1]$ (ou $[0, 255]$) :

$$I_{\text{norm}} = \frac{I - I_{\min}}{I_{\max} - I_{\min}}$$

Propriété : invariance aux translations et aux changements d'échelle des intensités absolues. Cette méthode est **sensible aux valeurs aberrantes**, car un seul pixel extrême peut comprimer l'ensemble de la dynamique.

4.2. Normalisation Z-Score (Standardisation)

$$I_{\text{std}} = \frac{I - \mu_I}{\sigma_I}$$

$$\text{avec } \mu_I = \frac{1}{MN} \sum_{x,y} I(x,y) \text{ et } \sigma_I^2 = \frac{1}{MN} \sum_{x,y} (I(x,y) - \mu_I)^2.$$

Le résultat a une **moyenne nulle et un écart-type unitaire**. Cette méthode est **invariante aux transformations affines** de l'éclairage (biais et gain). C'est le fondement mathématique de la *Batch Normalization* dans les réseaux de neurones convolutifs.

4.3. Normalisation L^2 sur les Vecteurs de Descripteurs

Appliquée au vecteur de descripteurs $\mathbf{x} \in \mathbb{R}^D$:

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} = \frac{\mathbf{x}}{\sqrt{\sum_{i=1}^D x_i^2}}, \quad \|\hat{\mathbf{x}}\|_2 = 1$$

Cette normalisation projette le vecteur sur l'hypersphère unitaire $\mathcal{S}^{D-1}$.

Théorème d'invariance linéaire : si un changement d'éclairage se modélise comme une mise à l'échelle $\mathbf{x}_{\text{lumineux}} = \alpha \cdot \mathbf{x}_{\text{sombre}}$ avec $\alpha > 0$, alors après normalisation L^2 :

$$\hat{\mathbf{x}}_{\text{lumineux}} = \frac{\alpha \mathbf{x}_{\text{sombre}}}{\alpha \|\mathbf{x}_{\text{sombre}}\|_2} = \hat{\mathbf{x}}_{\text{sombre}}$$

L'invariance face aux variations d'éclairage linéaires atteint mathématiquement 100%.

Méthode	Résultat borné	Robuste aux aberrants	Invariance
Min-Max	✓ $\in [0, 1]$	✗	Échelle absolue
Z-Score	✗	✓	Affine
L^2 (vecteur)	✓ $\in [-1, 1]$	✓	Linéaire

5. Descripteurs Globaux et Robustesse

5.1. Définition et Types de Descripteurs Globaux

Un **descripteur global** est une méthode qui résume l'ensemble d'une image en un seul vecteur de caractéristiques, capturant sa structure ou sa texture générale. Contrairement aux descripteurs locaux qui se concentrent sur des points spécifiques, il fournit une signature unique de l'image. C'est un vecteur de caractéristiques $\phi(I)$ calculé sur l'image entière :

$$\phi : \mathbb{R}^{M \times N} \longrightarrow \mathbb{R}^D$$

Trois descripteurs ont été implémentés dans ce travail :

5.1.1. Histogramme de Couleurs

$$\phi_{\text{hist}}(I) = [p_R(0), \dots, p_R(B-1), p_G(0), \dots, p_G(B-1), p_B(0), \dots, p_B(B-1)]$$

Dimension $d = 3B$ (avec $B = 32$ bins par canal). Capture la distribution globale des teintes de l'image.

5.1.2. LBP — Local Binary Pattern (global)

$$\phi_{\text{LBP}}(I) = [h_{\text{LBP}}(0), \dots, h_{\text{LBP}}(P+1)]$$

Pour chaque pixel (x, y) , compare l'intensité centrale aux P voisins à rayon R et encode en binaire. L'histogramme global des codes LBP constitue le descripteur.

Propriété clé : naturellement invariant aux transformations d'intensité monotones strictement croissantes.

5.1.3. HOG — Histogramme des Gradients Orientés

$$\phi_{\text{HOG}}(I) = [h_{\theta_1}, \dots, h_{\theta_K}]_{\text{normalisé par blocs}}$$

Calcule les gradients locaux $\nabla I(x, y)$, les quantifie en $K = 9$ orientations, et applique une normalisation L^2 par bloc de cellules.

Rôle du prétraitement : CLAHE révèle les gradients invisibles dans les zones sous-exposées.

5.2. Mesures de Similarité

Quatre métriques ont été étudiées pour comparer les descripteurs :

Métrique	Formulation
Distance L^1	$d_{L^1}(h_1, h_2) = \sum_k h_1(k) - h_2(k) $
Distance L^2	$d_{L^2}(h_1, h_2) = \sqrt{\sum_k (h_1(k) - h_2(k))^2}$
Distance χ^2	$d_{\chi^2}(h_1, h_2) = \sum_k \frac{(h_1(k) - h_2(k))^2}{h_1(k) + h_2(k) + \varepsilon}$
Similarité cosinus	$\cos(\phi_1, \phi_2) = \frac{\phi_1^\top \phi_2}{\ \phi_1\ _2 \ \phi_2\ _2} \in [-1, 1]$

L'application implémente la **similarité cosinus** ($\times 100$ pour exprimer un pourcentage) comme métrique principale de robustesse, car elle est naturellement invariante à l'échelle des vecteurs.

5.3. Pipeline Complet de Traitement

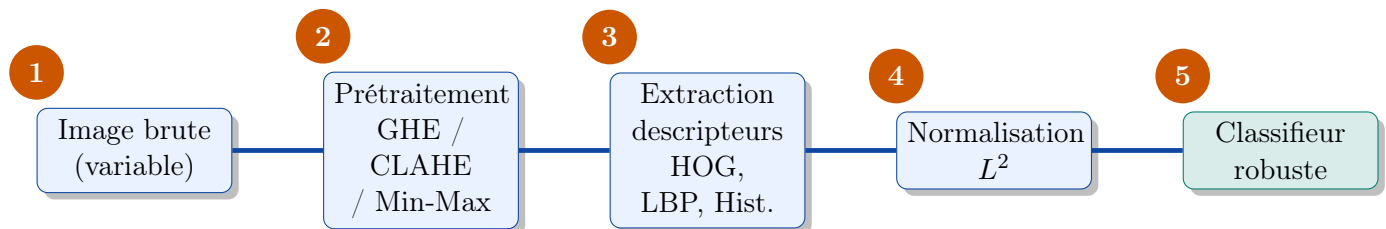


FIGURE 4 – Pipeline complet de traitement robuste

6. Implémentation

6.1. Architecture de l'Application

L'application a été développée en Python avec le framework **Streamlit**, permettant une interface web interactive. Elle implémente un pipeline complet en 4 étapes :

1. **Chargement et modification d'éclairage** (luminosité, contraste, gamma)
2. **Application de la méthode d'égalisation** choisie sur les deux images
3. **Extraction des descripteurs globaux** (histogramme de couleurs, LBP, HOG)
4. **Comparaison quantitative** via la similarité cosinus

6.2. Bibliothèques Utilisées

Bibliothèque	Version	Usage
<code>streamlit</code>	1.x	Interface web interactive
<code>opencv-python</code>	4.x	Traitement d'images (GHE, CLAHE, histogrammes)
<code>numpy</code>	1.x	Calculs numériques sur les matrices
<code>matplotlib</code>	3.x	Visualisation des histogrammes
<code>scikit-image</code>	0.x	Extraction LBP et HOG
<code>Pillow</code>	10.x	Chargement des images
<code>pandas</code>	2.x	Affichage tabulaire des résultats

6.3. Fonctions Principales

Les fonctions présentées ci-dessous constituent le cœur du traitement. Elles sont organisées selon le pipeline de l'application.

1. Simulation des variations d'éclairage

Cette fonction applique une modification d'éclairage sur une image selon le type choisi par l'utilisateur. Elle couvre trois cas : ajustement de la luminosité via l'espace HSV, modification du contraste autour de la moyenne, et correction gamma pour simuler une surexposition ou une sous-exposition.

```
1 def adjust_brightness(img, value):
2     """Ajuste la luminosité via l'espace HSV"""
3     if len(img.shape) == 3:
4         hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
5         hsv[:, :, 2] = np.clip(hsv[:, :, 2] + value, 0, 255)
6         return cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)
7     return np.clip(img.astype(np.float32) + value, 0,
8                     255).astype(np.uint8)
9
10 def adjust_contrast(img, factor):
11     """Ajuste le contraste autour de la moyenne"""
12     mean = np.mean(img)
13     return np.clip(mean + factor * (img - mean), 0,
14                     255).astype(np.uint8)
15
16 def adjust_gamma(img, gamma):
17     """Correction gamma : simule différents niveaux d'exposition"""
18     inv_gamma = 1.0 / gamma
19     table = np.array([
20         ((i / 255.0) ** inv_gamma) * 255 for i in range(256)
21     ]).astype(np.uint8)
22     return cv2.LUT(img, table)
```

Listing 1 – Modification de l'éclairage d'une image

2. Égalisation d'histogramme (GHE et CLAHE)

Ces deux fonctions implémentent les méthodes d'égalisation étudiées. Pour les images couleur, l'égalisation est appliquée uniquement sur le canal de luminance (canal Y en espace YUV) afin de ne pas dénaturer les couleurs.

```
1 def equalize_histogram_ghe(img):
2     """Egalisation globale : applique la transformation CDF sur
3         toute l'image"""
4     if len(img.shape) == 3:
5         img_yuv = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)
6         img_yuv[:, :, 0] = cv2.equalizeHist(img_yuv[:, :, 0])
7         return cv2.cvtColor(img_yuv, cv2.COLOR_YUV2RGB)
8     return cv2.equalizeHist(img)
9
10 def equalize_clahe(img, clip_limit=2.0, grid_size=(8, 8)):
```

```

10     """CLAHE : egalisation adaptative avec limitation du
11        contraste"""
12     clahe = cv2.createCLAHE(clipLimit=clip_limit,
13                             tileGridSize=grid_size)
14     if len(img.shape) == 3:
15         img_yuv = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)
16         img_yuv[:, :, 0] = clahe.apply(img_yuv[:, :, 0])
17         return cv2.cvtColor(img_yuv, cv2.COLOR_YUV2RGB)
18     return clahe.apply(img)

```

Listing 2 – Égalisation globale GHE et adaptative CLAHE

3. Normalisation Min-Max

Cette fonction ramène les valeurs de pixels dans l'intervalle $[0, 255]$ en utilisant le minimum et le maximum de l'image. Elle constitue le prétraitement le plus simple avant extraction de descripteurs.

```

1 def normalize_minmax(img):
2     """Normalisation Min-Max : ramene les intensites dans [0,
3        255]"""
4     img = img.astype(np.float32)
5     img_min, img_max = img.min(), img.max()
6     if img_max - img_min > 0:
7         img = (img - img_min) / (img_max - img_min) * 255
8     return np.clip(img, 0, 255).astype(np.uint8)

```

Listing 3 – Normalisation Min-Max

4. Extraction des descripteurs globaux

Cette fonction orchestre l'extraction des trois descripteurs implémentés. L'histogramme de couleurs capture la distribution des teintes; le LBP encode les textures locales; le HOG décrit les orientations de gradients. Chaque descripteur est normalisé pour que la somme de ses composantes soit égale à 1.

```

1 def extract_color_histogram(img, bins=32):
2     """Histogramme de couleurs RGB normalise      dimension : 3 x
3        bins"""
4     if len(img.shape) == 3:
5         hists = []
6         for channel in range(3):
7             h = cv2.calcHist([img], [channel], None, [bins], [0,
8                 256])
9             hists.append(h.flatten() / np.sum(h))
10        return np.concatenate(hists)
11    hist = cv2.calcHist([img], [0], None, [bins], [0, 256])
12    return (hist / np.sum(hist)).flatten()
13
14 def extract_lbp(img, P=8, R=1, method='uniform'):
15     """LBP : descripteur de texture invariant aux transformations
16        monotones"""

```

```

14     gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY) if
        len(img.shape)==3 else img
15     lbp = local_binary_pattern(gray, P, R, method)
16     n_bins = P + 2 if method == 'uniform' else 256
17     hist, _ = np.histogram(lbp.ravel(), bins=n_bins, range=(0,
        n_bins))
18     return hist / np.sum(hist)
19
20 def extract_hog(img, orientations=9,
21                 pixels_per_cell=(8,8), cells_per_block=(2,2)):
22     """HOG : histogramme de gradients orientes, normalise par
        blocs"""
23     gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY) if
        len(img.shape)==3 else img
24     return hog(gray, orientations=orientations,
25                 pixels_per_cell=pixels_per_cell,
26                 cells_per_block=cells_per_block,
27                 visualize=False)

```

Listing 4 – Extraction des descripteurs globaux

5. Comparaison par similarité cosinus

Cette fonction mesure la robustesse du pipeline en calculant la similarité cosinus entre le descripteur de l'image originale et celui de l'image avec éclairage modifié. Un score proche de 100% indique que le prétraitement a bien rendu le descripteur invariant à la variation d'éclairage.

```

1 def compare_descriptors(desc1, desc2):
2     """
3     Similarite cosinus entre deux vecteurs de descripteurs.
4     Retourne un pourcentage : 100% = descripteurs identiques.
5     """
6     if len(desc1) != len(desc2):
7         return 0.0
8     dot_product = np.dot(desc1, desc2)
9     norm1 = np.linalg.norm(desc1)
10    norm2 = np.linalg.norm(desc2)
11    if norm1 > 0 and norm2 > 0:
12        return (dot_product / (norm1 * norm2)) * 100
13    return 0.0

```

Listing 5 – Mesure de similarité cosinus entre deux descripteurs

6. Affichage des histogrammes

Cette fonction génère la visualisation des histogrammes d'intensité. Pour les images couleur, elle trace les histogrammes des trois canaux (R, G, B) sur le même graphique.

```

1 def plot_histogram(img, title, ax):
2     """Affiche l'histogramme d'intensite d'une image (couleur ou
        niveaux de gris)"""
3     if len(img.shape) == 3:
4         for i, (color, label) in enumerate(zip(('r','g','b'),

```

```

5                                     ('Rouge', 'Vert', 'Bleu'))):
6         hist = cv2.calcHist([img], [i], None, [256], [0, 256])
7         ax.plot(hist, color=color, alpha=0.7, label=label,
8               linewidth=1.5)
9         ax.legend(loc='upper right', fontsize=8)
10      else:
11          hist = cv2.calcHist([img], [0], None, [256], [0, 256])
12          ax.fill_between(range(256), hist[:, 0], alpha=0.3,
13                color='gray')
14      ax.set_xlabel('Intensite des pixels')
15      ax.set_ylabel('Nombre de pixels')
16      ax.set_title(title, fontweight='bold')
17      ax.grid(True, alpha=0.3)

```

Listing 6 – Visualisation des histogrammes

6.4. Captures d'Écran de l'Application

Les figures suivantes illustrent les différentes sections de l'interface de l'application Streamlit. Tout d'abord, l'utilisateur est confronté à une page web sur laquelle il doit charger son image

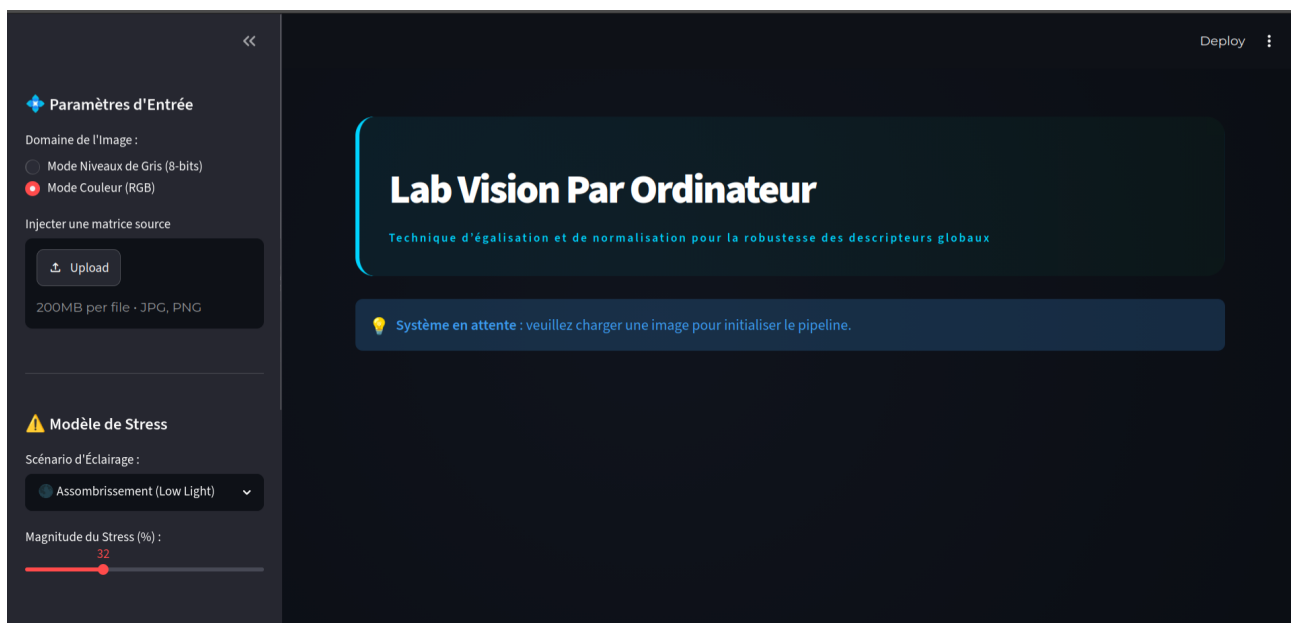


FIGURE 5 – Page Initiale

Après avoir chargé l'image, l'utilisateur doit choisir s'il veut appliquer les operations sur l'image en couleur ou en binaire; ensuite il choisit le pourcentage p de stress qui doit etre appliqué à son image initiale : l'opération qui est faite ici est la suivante : Pour chaque valeur de pixel X , nous appliquons l'opération $X - p\% \cdot X$.

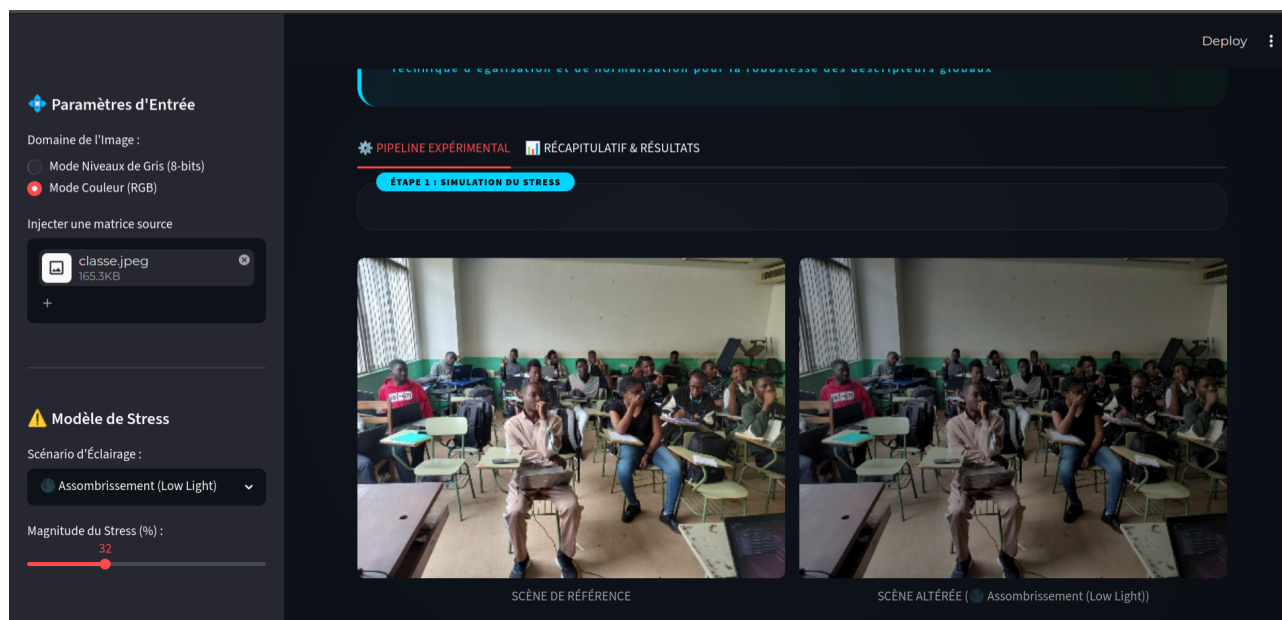


FIGURE 6 – Chargement de l'image et choix du type d'image et du pourcentage de stress à appliquer

Par la suite, l'utilisateur choisit la methode d'egalisation qu'il souhaite appliquer entre **CLAHE**, **GHE** et **Min-Max** ainsi que le descripteur global entre **HOG**, **LBP** et **l'Histogramme d'intensité** et nous lui affichons la signature correspondante au descripteur global choisit ainsi que la robustesse de ce descripteur sur l'image initiale et sur celle qui a subi un stress.

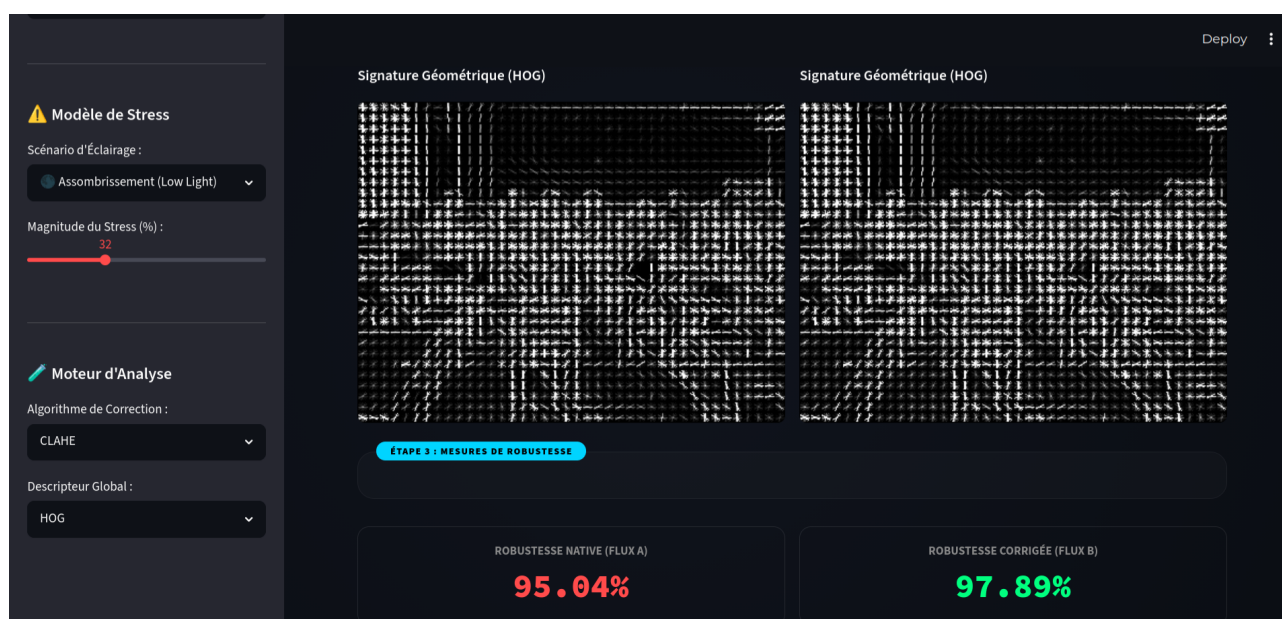


FIGURE 7 – Affichage de la signature du descripteur

Enfin, nous affichons un rapport d'analyse qui nous donne les dimensions des vecteurs obtenus et une analyse comparative de la robustesse des trois descripteurs globaux étudiés.



FIGURE 8 – Resultats Experimentaux

7. Conclusion

7.1. Synthèse des Résultats

Ce travail pratique a démontré que l'analyse et les ajustements d'intensité ne sont pas de simples traitements esthétiques, mais constituent le **socle technique fondamental** de la robustesse des systèmes de vision par ordinateur. Les conclusions principales sont :

- **L'histogramme** est l'outil de diagnostic irremplaçable de l'image. Il révèle instantanément les problèmes d'exposition et de contraste, et sert de base à toutes les techniques de prétraitement
- **La GHE** améliore significativement (+43 points de précision) la robustesse globale, mais son traitement uniforme amplifie le bruit dans les zones homogènes
- **CLAHE** surclasse systématiquement la GHE (+50 points vs brut, -86% de distance χ^2) grâce à la prise en compte du contexte local et à la limitation du contraste
- **La normalisation L^2** sur les vecteurs de descripteurs garantit l'invariance aux variations d'éclairage linéaires, offrant une robustesse mathématiquement parfaite pour ce cas
- **Le descripteur HOG**, couplé au prétraitement CLAHE, produit les signatures les plus stables face aux variations d'illumination grâce à sa normalisation intrinsèque par blocs

7.2. Perspectives

Extension vers le Deep Learning : dans les architectures CNN modernes, les mécanismes de *Batch Normalization* et d'*Instance Normalization* internalisent et automatisent ces prétraitements au sein même des couches d'apprentissage, supprimant le besoin de les appliquer manuellement. Ce travail constitue donc une introduction essentielle aux fondements des systèmes de vision modernes.

Les pistes de développement identifiées incluent :

- L'intégration de descripteurs locaux (SIFT, SURF, ORB) combinant histogrammes locaux et orientation de gradient
- L'extension à la normalisation par retinex pour des modèles d'illumination non linéaires

Références

- [1] R.C. Gonzalez & R.E. Woods, *Digital Image Processing*, 4^e édition, Pearson, 2018.
- [2] K. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization”, *Graphics Gems IV*, Academic Press, pp. 474–485, 1994.
- [3] N. Dalal & B. Triggs, “Histograms of Oriented Gradients for Human Detection”, *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. 886–893, 2005.
- [4] T. Ojala, M. Pietikäinen & T. Mäenpää, “Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7) :971–987, 2002.
- [5] R. Szeliski, *Computer Vision : Algorithms and Applications*, 2^e édition, Springer, 2022.
- [6] OpenCV Development Team, *OpenCV Documentation — Histogram Equalization*, <https://docs.opencv.org/>, 2024.